

Multi-Client File Transfer Protocol

Protocol Specification Document

1. Overview

This document specifies a custom application-layer protocol, referred to as MCFTP (Multi-Client File Transfer Protocol), built on top of TCP sockets. MCFTP supports user authentication, private and broadcast messaging, direct file upload/download to the server, and file transfer between clients mediated by the server. The server is multi-threaded and supports concurrent client sessions.

2. Transport and Message Framing

All communication occurs over TCP. Since TCP delivers a byte stream with no inherent message boundaries, MCFTP uses two framing strategies depending on content type:

- Control commands** (text): newline-terminated ASCII strings, encoded UTF-8. Fields are separated by single spaces. Every command line ends in `\n`.
- File payloads** (binary): every command that transfers a file first sends the file size in bytes as part of the command line. The receiver then performs a length-bounded read loop, calling `recv( )` repeatedly and accumulating bytes until exactly `size` bytes have been read (a single `recv( )` call is not guaranteed to return the full payload).

3. Command Reference

Commands are sent Client -> Server unless otherwise noted. Responses follow the format described in Section 5.

Command	Direction	Purpose
LOGIN <username>	C -> S	Authenticate with a unique username. No password in this iteration (per assignment scope).
LOGOUT	C -> S	Graceful disconnect; server removes user from active-users map.
LIST	C -> S	Request the list of currently active usernames.
MSG <user> <text>	C -> S -> C	Private message; server routes to the named user if online.
BROADCAST <text>	C -> S -> C*	Message delivered to every currently connected client.
UPLOAD <filename> <size>	C -> S	Store a file of <size> bytes in the sender's server-side storage.
DOWNLOAD <filename>	C -> S	Retrieve a previously uploaded file from the server.
SEND <user> <filename> <size>	C -> S -> C	Transfer a file to another user via the server (see Section 6).
INCOMING <user> <filename> <size>	S -> C	Server push notifying a client that a file follows (from SEND or pending delivery).

Example command line (raw bytes on the wire)

5. Server Response Format

Every command produces exactly one immediate response line from the server, in addition to any asynchronous pushes (e.g. INCOMING, forwarded MSG/BROADCAST). Two response types are defined:

OK <info...>
ERR <code> <reason>

Example success: OK LOGIN alice. Example failure: ERR 002 FILE_NOT_FOUND.

Code	Meaning
001	USERNAME_TAKEN - login rejected, username already active
002	FILE_NOT_FOUND - requested file does not exist on server
003	USER_OFFLINE - target user for MSG/SEND is not currently connected and unknown to the server this session
004	MALFORMED_COMMAND - command could not be parsed
005	UNAUTHENTICATED - command received before a successful LOGIN

6. SEND Semantics: Offline-Target Delivery

Unlike MSG (which is rejected outright if the target is offline), SEND guarantees eventual delivery. The server maintains a `pending_transfers` structure: a dictionary mapping `username -> list of (sender, filename, filepath, size, timestamp)`, guarded by the same lock used for the active-users map.

- **Target online:** the server reads the incoming file from the sender and immediately streams it to the target's live socket as an unsolicited INCOMING push.
- **Target offline or unknown:** the server still accepts the upload from the sender (replying `OK SEND_QUEUED`) and writes the file to `server_storage/pending/<target>/<filename>`, recording an entry in `pending_transfers`.
- **Delivery on login:** when the target later sends a successful LOGIN, the server checks `pending_transfers` for that username immediately after authentication succeeds. Every queued file is streamed to the client as an INCOMING push, and its entry is removed once fully sent.

Pending storage is kept separate from a user's own UPLOAD/DOWNLOAD directory to avoid filename collisions between a user's personal files and files queued for delivery to them.

7. Sequence Diagrams

Sequence 1: LOGIN (success and duplicate-username rejection)

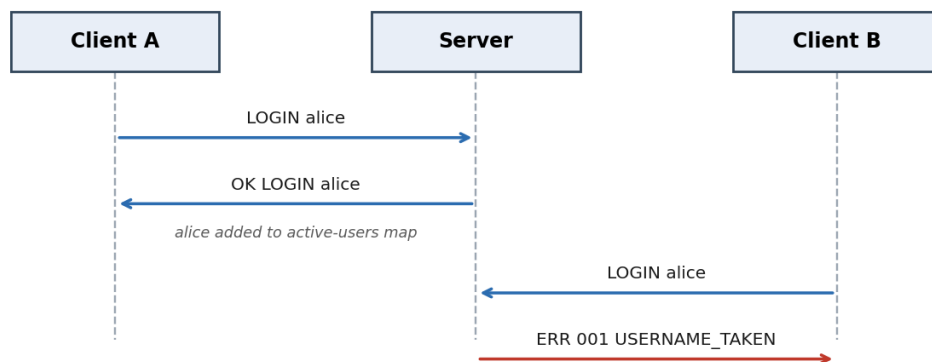


Figure 1. LOGIN: successful authentication, followed by a rejected duplicate-username attempt.

Sequence 2: Private MSG between two online users

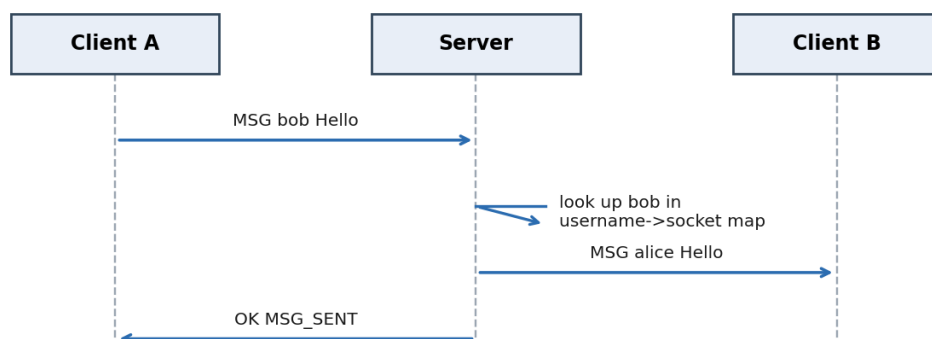


Figure 2. Private MSG delivered between two online clients via server-side routing.

Sequence 5: UPLOAD and DOWNLOAD

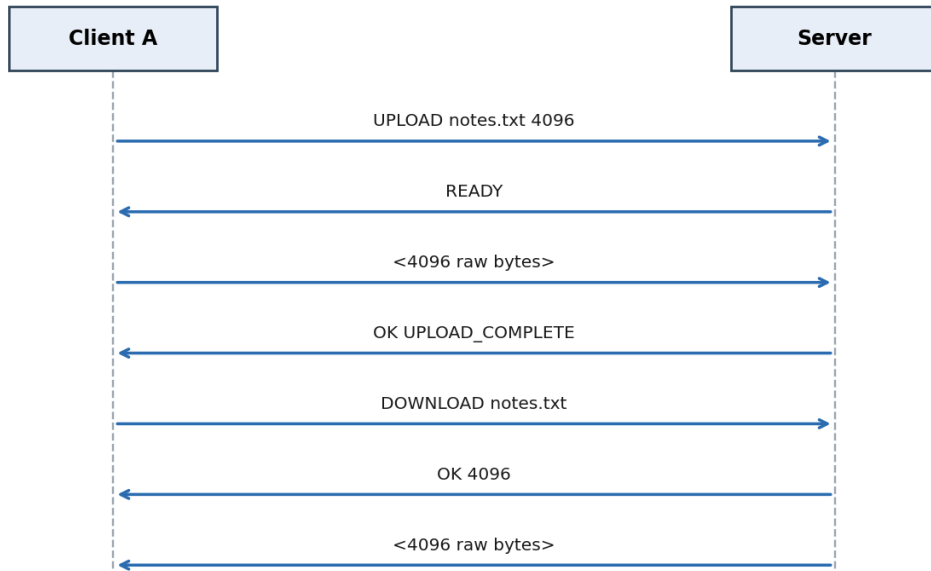


Figure 3. UPLOAD followed by DOWNLOAD of the same file.

Sequence 3: SEND - target user online (live forward)

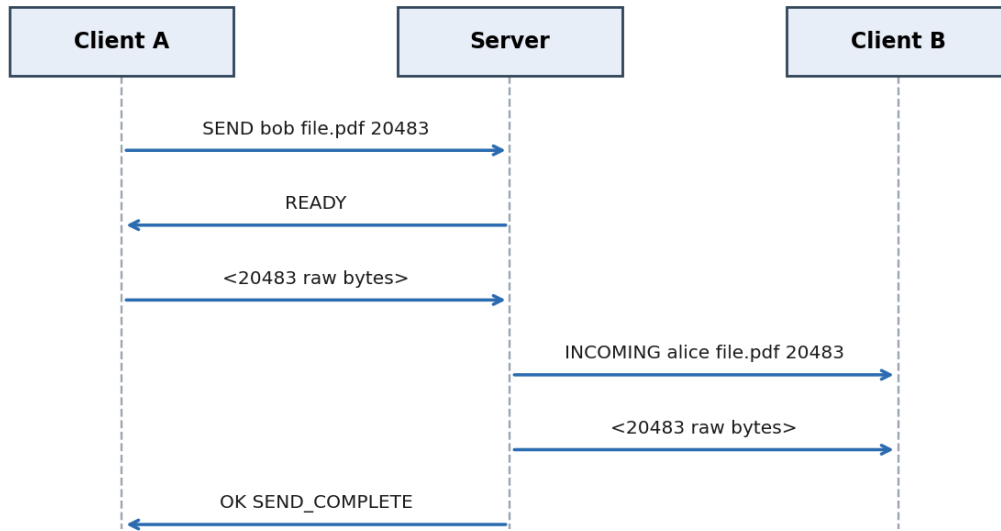


Figure 4. SEND when the target user is online: server forwards the file live.

Sequence 4: SEND - target offline, auto-delivered on next LOGIN

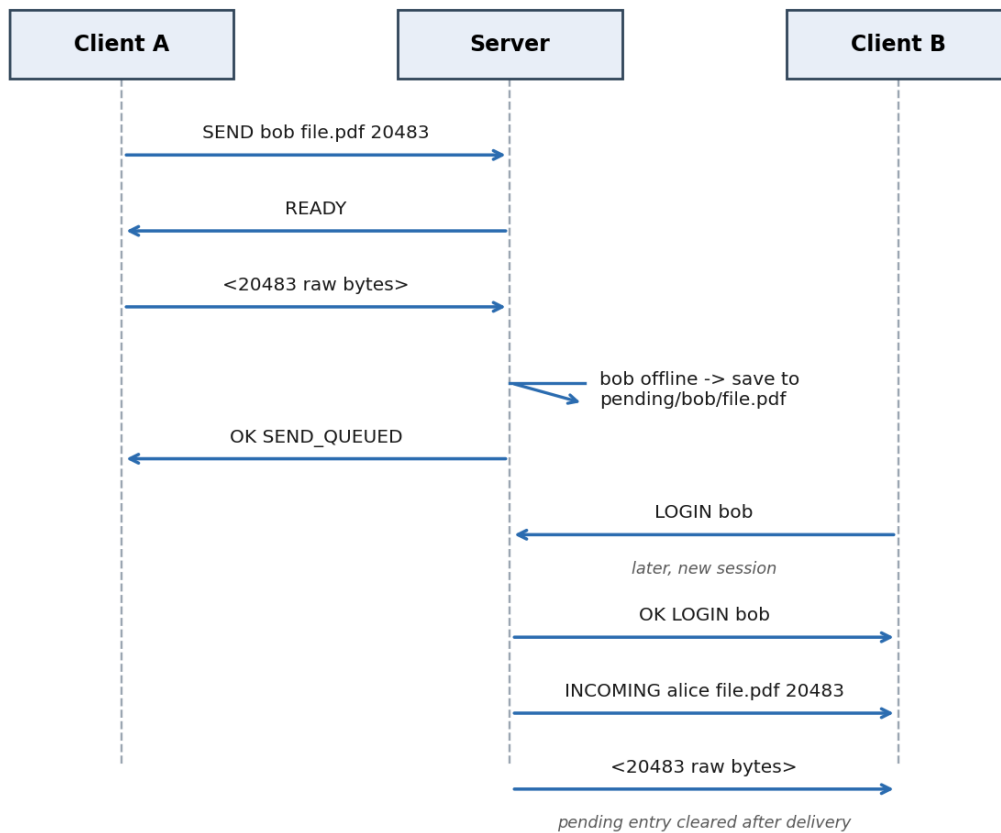


Figure 5. SEND when the target is offline: file is queued and auto-delivered on the target's next LOGIN.

8. Concurrency Model (summary)

The server accepts connections on the main thread and spawns one worker thread per client connection. Shared mutable state - the active-users map (username -> socket), the pending-transfers map, and the on-disk file storage - is protected by a single threading . Lock (or a small set of per-structure locks) to prevent race conditions when multiple client threads read or modify them concurrently, e.g. two clients logging in with the same username at the same instant, or a BROADCAST iterating the active-users map while another thread is removing a disconnecting client.